DEV400

Time Series

MongoDB Developer Extension

Release: 20260220

Topics we cover

What is time-series?

How does MongoDB incorporate time-series?

Configuration options

Querying time-series

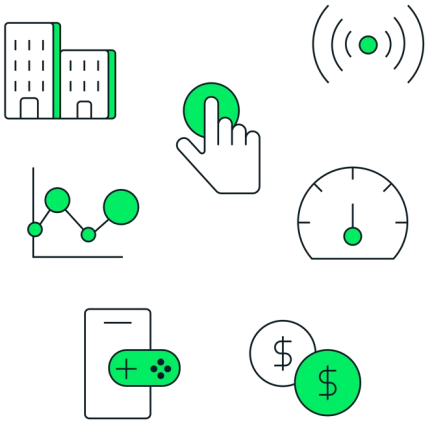
Intro to Time Series Collections

Time Series Collections are not directly related to Indexing but they help us optimize storage and retrieval of data that specifically deals with time (or time series). They use indexing in the background and also require some basic knowledge of a bucketting pattern. We will cover all the necessary topics with a view to understanding Time Series Collections better.

What is Time-Series in general?

Time Series is any system that accumulates a large number of **data points** and usually wants to query it by **time slices**, like IoT device data, stock trading, clickstreams, monitoring (DevOps), etc.

In essence, **time-series data** is a set of measurements taken at **time intervals**. The queries to this data include a time component.



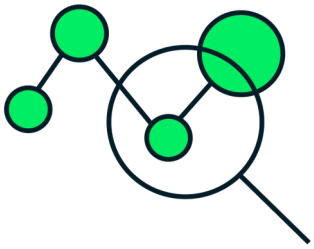
The Key is that we aren't looking at one record but at a number of them covering a specific time period. Time series is about looking at how data changes, rather than a snapshot of data. There is a growing popularity of such kind of systems and hence a demand of storing such data efficiently.

How does MongoDB help?

Time Series Collections in **MongoDB** makes the gathering, storage and analysis of time-series data faster, easier and more efficient.

It leverages **advanced indexing**, **granular query controls**, and a **columnar storage format** to **autonomously manage bucketing** and seamlessly integrate with other MongoDB services, **optimizing performance** and **reducing storage costs**.

Note: We don't need to know the exact meaning of all these terms right now.



Note: One doesn't need to know every aspect of the internals of time series to start learning (and even implementing) Time Series Collections. We would be covering the required topics as and when necessary.

In MongoDB normally each document is stored separately and if one wants to co-locate data, the developer has to generate these combined records.
In Time series - additional info, when creating the collection, allows the database to reorganize data behind the scenes, to co-locate relevant data, and apply additional compression and indexing allowing it to be retrieved faster.

Use case: BTC Ticker

Analyzing the Data

Download the JSON

```
curl -O https://ps-ilt-timeseries-assets.s3.eu-west-1.amazonaws.com/btc2022.json
```

Restore the archive

Execute a findOne query

Check the structure

Though it contains time-series data, this is **NOT** a time series collection.

```
> curl -O https://ps-ilt-timeseries-assets.s3.eu-west-1.amazonaws.com/btc2022.json
> mongoimport 'mongodb+srv://<user>:<password>@cluster0.a.mongodb.net' --collection=btc2022 --db=test --file=btc2022.json
> mongosh "mongodb+srv://cluster0.a.mongodb.net/test" --username admin

> use test
> db.btc2022.findOne()
{
  _id: ObjectId("..."),
  datetime: ISODate("2022-12-30T00:00:00.000Z"),
  symbol: 'BTCUSD',
  open: 16636.4,
  high: 16644.4,
  low: 16360,
  close: 16607.2,
  vol: '192.76K',
  change_percent: '-0.18%'
}
```


This document shows us the crypto market related data of BitCoin vs USD. The fields open, high, low, close represent the opening price, highest price, lowest price and closing price respectively of one BitCoin on that day (essentially a candlestick). The entire btc2022 collection consists of the data of symbols BTCUSD, BTCGBP and BTCAUD for the year 2022 only. This amounts to roughly 1K documents.

Command to download the JSON

curl -O https://ps-ilt-timeseries-assets.s3.eu-west-1.amazonaws.com/btc2022.json

The collection btc2022 is just used for data storage purposes, so that we can insert data into the actual time series collection later.

The Problem

Imagine if we store the data for all currencies (instead of just three) and that too for all the years (instead of just 2022).

Things would start getting more resource intensive when we start storing hourly data instead of daily data.

Even more difficulties can be faced with minute-wise data.

```
{
  _id: ObjectId("6410ce6aca9608a8fb00eee4"),
  datetime: ISODate("2022-12-30T00:00:00.000Z"),
  symbol: 'BTCUSD',
  open: 16636.4,
  high: 16644.4,
  low: 16360,
  close: 16607.2,
  vol: '192.76K',
  change_percent: '-0.18%'
}
```


Sometimes time-series data will come into your database at high frequency - use-cases like financial transactions, stock market data, readings from smart meters, or metrics from services you're hosting over hundreds or even thousands of servers. In other cases, each measurement may only come in every few minutes. Maybe you're tracking the number of servers that you're running every few minutes to estimate your server costs for the month. Perhaps you're measuring the soil moisture of your favourite plant once a day.

The Question

How would we be able to query the prices by **datetime** or **symbol** or **both** in such a big collection?

```
{
  _id: ObjectId("6410ce6aca9608a8fb00eee4"),
  datetime: ISODate("2022-12-30T00:00:00.000Z"),
  symbol: 'BTCUSD',
  open: 16636.4,
  high: 16644.4,
  low: 16360,
  close: 16607.2,
  vol: '192.76K',
  change_percent: '-0.18%'
}
```


We can create a compound index, but then, imagine the size of the index.
The primary concern of various MongoDB users who use indexing rather than bucketing is that the indexes get very large in size for small documents which are plenty in volume.
It requires a more sophisticated solution than merely indexing.

The Solution - A Time Series Collection

Here the **timeField** option is required and it should have the timestamp (BSON date) to which our individual documents correspond.

In this case it is **datetime.**

What does our use case need apart from datetime so that queries are more efficient?

```
> db.createCollection(
    "btc_ticker",
    {
       timeseries: {
          timeField: "datetime",
       }
    }
)

{ ok: 1 }
```


Though this is enough to create a time series collection, our case requires the data to be categorized by symbol name.

Follow Along Exercise - the metaField

We need to **label** our price data with a **symbol**

In our case, symbol uniquely identifies the price on a particular **datetime**.

metaField should be the one that doesn't change or rarely changes.

metaField is virtually a **secondary index** in most of the use cases.

```
> db.btc_ticker.drop()

> db.createCollection(

    "btc_ticker",
    {
       timeseries: {
         timeField: "datetime",
   metaField: "symbol"
       }
    }
)

// metaField can also refer to embedded documents
```


metaField can also refer to an embedded document in the source data.
For instance metaData:{symbol:'BTCUSD', exchange:'COINBASE'}

Three objects get created

The **`btc_ticker`** collection which is actually a **writable view**.

`systems.buckets.btc_ticker` where the **actual data** is stored.

`system.views` maps the time series collection with its underlying bucket collection.

Do you think that **`systems.buckets.btc_ticker`** would have any data?

```
> show collections

btc_ticker                    [time-series]

btc2022

system.buckets.btc_ticker

> db.system.buckets.btc_ticker.find()
```


The systems.buckets.btc_ticker would not have any data unless we start inserting some in the btc_ticker time-series collection.

Follow Along Exercise - Inserting data

Insert operation is done as usual.

What output would you expect from the **`btc_ticker`** collection?

Which collection would be more interesting to check at this point?

```
> db.btc2022.find().forEach(function(doc){

   db.btc_ticker.insertOne(doc);

});

> db.btc_ticker.find()
```


In this case we are just inserting data from btc2022 collection that we imported earlier. In real-world, the data might come from an application in real time.

The documents should be inserted in time order (due to the nature of use cases in which these collections are used) usually and that would be a better structure for time series collection.

The btc_ticker collection would would have the same data as btc2022. The more interesting collection would be system.buckets.btc_ticker though.

The Actual Buckets of Data

As you can see, there are buckets formed in our actual collection with a label (**meta**) attached.

These buckets contain the range of data from **min** to **max.**

The combination of meta and min/max help MongoDB retrieve our data fairly quickly and organize our data better.

```
> db.system.buckets.btc_ticker.findOne()

{
  _id: ObjectId("61cf9980d6fa11728e2afd87"),
  control: {
    version: 1,
    min: {
      _id: ObjectId("6410ce6aca9608a8fb00f04b"),
      datetime: ISODate("2022-01-01T00:00:00.000Z"),
      ...
    },
    max: {
      _id: ObjectId("6410ce6aca9608a8fb00f04b"),
      datetime: ISODate("2022-01-01T00:00:00.000Z"),
      ...
    }
  },
  meta: 'BTCUSD',
  data: {
    vol: { '0': '31.24K' },
    high: { '0': 47917.6 },
    datetime: { '0': ISODate("2022-01-01T00:00:00.000Z") },
    open: { '0': 46217.5 },
    close: { '0': 47738 },
    change_percent: { '0': '3.29%' },
    _id: { '0': ObjectId("6410ce6aca9608a8fb00f04b") },
    low: { '0': 46217.5 }
  }
}
```


The entire complex process of showing the data in our view from the buckets of data is carried out by MongoDB internally.

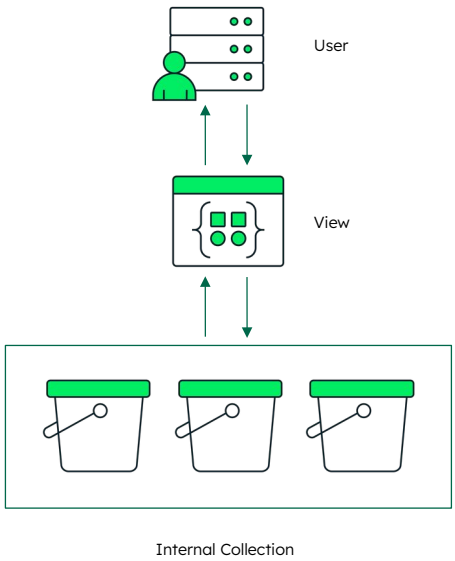
The formal definition

A time-series collection is an abstract layer containing a writable **non-materialized view** on top of an **internal (bucket) collection**.

The user just has to work with the above mentioned **abstract layer** while MongoDB takes care of the internal complications.

On the surface, time series collections behave **exactly** like normal collections except that they are more efficient.

We can **insert and query** our data as one normally would.



Under the hood, the creation of a time series collection results in a collection and an automatically created writable non-materialized view which serves as an abstraction layer. This abstraction layer allows you to always work with their data as single documents in their raw form without worry of performance implications as the actual time series collection implements a form of the bucket pattern when persisting data to disk, but these details are something you **no longer need to care** about when designing your schema or reading and writing your data. Users will always be able to work with the abstraction layer and not with a complicated compressed bucketed document.

What Next?

After this introduction to time series collections, there is more that lies ahead. The topics that can be explored are:

- Granularity
- Window Functions
- Expiration (TTL) in time series
- Sharding a time series collection

and much more.

https://www.mongodb.com/developer/products/mongodb/window-functions-and-time-series/
The true value of time series comes from being able to do complex things like an average of the past 5 days.

Quiz Time!

#1. Which of the following might be suitable for a time-series collection?

A	Stock Data	B	Data on Store Inventory	C	Website Visitors
D	Geographical Location Data	E	Weather Data		

Answer in the next slide.

#1. Which of the following might be suitable for a time-series collection?

A Stock Data

B Data on Store Inventory

C Website Visitors

D Geographical Location Data

E Weather Data

Time series data is most suitable for data that can be looked at over a span of time or time slices. If the data itself is going to be keeping track of locations or managing inventory without a correlation to time, it might not be the best use case for it.

#2. Which option is invalid when constructing a time-series collection?

A	timeField	B	filterField	C	Granularity
D	expireAfterSeconds	E	metaField		

Answer in the next slide.

#2. Which option is invalid when constructing a time-series collection?

A timeField

B filterField

C Granularity

D expireAfterSeconds

E metaField

Time series can accept many different arguments when configuring a collection or modifying an existing one - the one parameter however that does not exist is the filterField, all other options seen here do exist.

#3. How can granularity be configured on an existing time-series collection?

A	Modifying it against increasing timespans
B	Modifying it against decreasing timespans
C	Recreate collection with new granularity option
D	Using the expireAfterSeconds option
E	Can't be configured once set

Answer in the next slide.

#3. How can granularity be configured on an existing time-series collection?

A	Modifying it against increasing timespans
B	Modifying it against decreasing timespans
C	Recreate collection with new granularity option
D	Using the expireAfterSeconds option
E	Can't be configured once set

Granularity can only be set on increasing timespans so from seconds → minutes → hours. It cannot be set in any other way. The default when not specifying this property is seconds.

Recap

Writes are organized so that data from the same source are stored alongside other data points from a similar point in time.

Time series provides benefits such as improved query efficiency, reduced disk usage & reduced I/O.

Time series collections behave like normal collections in terms of querying and inserting data.